Reviewer Pack — Minimal Rank of Tropicalized Configuration Spaces vs ACC⁰ Ci…

Entry dc98e63645dd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 13:36:01 UTC

Minimal Rank of Tropicalized Configuration Spaces vs ACC⁰ Circuit Threshold for Tensor Product

Entry ID: `dc98e63645dd` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 13:36:01 UTC

1. Conjecture

Field A (mathematical branch): Configuration Space Theory **Field B** (complexity object): Complexity Theory: ACC⁰ Circuit Complexity

Statement:

> For every Boolean function f: {0,1}^n -> {0,1}, let T_f denote the tropicalized configuration space of the set {f(x) | x ∈ {0,1}^n}. Then the minimal rank of T_f is upper-bounded by the ACC⁰ circuit threshold for the tensor product of f and its negation, i.e., min_r rank(T_f) ≤ θ(n, |f|), where θ(n, k) is an increasing function that grows slower than $2^{n/2}$k.

Rationale (proposer's reasoning):

> Configuration space theory has shown promise in understanding geometric properties of complex systems. Tropicalization offers a computational tool for analyzing these properties over the tropical semiring. By connecting this with ACC⁰ circuit complexity, we aim to expose new insights into the structure of Boolean functions and their computation.

Taxonomy category: `Configuration_Space_Theory_xACC0_Circuit_Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8b7441693e26f7fe`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given Boolean function f: {0,1}^n -> {0,1}, if the minimal rank of its tropicalized configuration space T_f is less than or equal to the ACC⁰ circuit threshold for the tensor product of f and its negation, specifically if min_r(rank(T_f)) ≤ θ(n, |f|), then the conjecture is supported. Otherwise, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 4 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropicalized configuration space" AND "ACC⁰ circuit complexity" - "minimal rank" AND "tensor product" AND "Boolean function" - "Configuration Space Theory" AND "Complexity Theory: ACC⁰ Circuit Threshold"

Top relevant hits considered: - [s2:10.1109/ACCESS.2025.3638801] Design and Non-Intrusive Nearfield Calibration of a Low-Complexity, Modular, Wideband, and Circularly Polarized Active E - [s2:4525865e1ea6ceca6557f09b40eb657485598ecf] Fast Circuit Satisfiability Algorithms and Applications - [s2:10.1007/s00020-014-2139-8] Tensor Products of Subspace Lattices and Rank One Density - [s2:2307.04538] Braiding and asymptotic Schur's orthogonality

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 10  # Start with a small size and increase if needed
    while True:
        f = [random.randint(0, 1) for _ in range(2**n)]
        T_f = []

        for x in range(2**n):
            T_f.append(f[x])

        # Compute the ACC⁰ circuit threshold for the tensor product of f and its negation
        theta_n_k = (2**n) / 2**(len(f))

        # Measure the minimal rank of T_f
        min_rank = len(set(T_f))

        if min_rank <= theta_n_k:
            conjecture_holds = True
            counterexample = ""
        else:
```

```
            conjecture_holds = False
            counterexample = f"min_rank={min_rank} > theta_n_k={theta_n_k}"

        return {
            "metric_name": "minimal_rank",
            "metric_value": min_rank,
            "instances_tested": 1,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = results[first_failing_seed]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
077778436e-306'}
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':
306'}
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':
306'}
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':
306'}
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':
306'}
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':
306'}
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':
306'}
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':
306'}
```

```
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':
306'}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9cac8ca2.py", line 69, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9cac8ca2.py", line 69, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                                ~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results show that for at least one instance, the minimal rank of the tropicalized configuration space exceeds the ACC^0 circuit threshold for | next: Investigate further to identify the specific Boolean function and its negation that provides a counterexample. This will help refine the conjecture or develop a new one.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15254
2	propose	ollama_remote	glm4:latest	0	0	13204
3	preregistration	ollama_remote	glm4:latest	0	0	9819
4	novelty	ollama_remote	glm4:latest	0	0	8323
5	novelty	ollama_remote	glm4:latest	0	0	9124
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12056
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13026
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10508
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6633
10	judge	ollama_remote	glm4:latest	0	0	28470

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 126416 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/dc98e63645dd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dc98e63645dd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/dc98e63645dd.tar.gz` (if generated)